

VENTAS FRONTEND · GUÍA DE ESTUDIO

HTML, CSS y el DOM

Glosario completo del proyecto

Todas las etiquetas, atributos y propiedades CSS que usa la app
+ cómo trabajan juntos + preguntas típicas de entrevista

Angular 21

CSS + Flexbox + Grid

DOM

Proyecto: Ventas-Frontend (demo-ira-car) · Preparación entrevista fullstack

Índice

- 1 Cómo se conectan HTML, CSS y JavaScript (y Angular)
- 2 El DOM (Document Object Model)
- 3 HTML: etiquetas y atributos usados en el proyecto
- 4 CSS: fundamentos, cascada, modelo de caja, unidades y colores
- 5 Glosario de propiedades CSS (por categoría, con ejemplos del proyecto)
- 6 Ejemplos reales de la app explicados paso a paso
- 7 Preguntas típicas de entrevista con respuestas
- 8 Cheat-sheet de un vistazo

Cómo leer este documento: cada propiedad y etiqueta incluye **qué hace** y un **ejemplo real del código** de Ventas-Frontend (entre etiquetas `<code>`). Puedes saltar directo a la sección 5 si solo quieres repasar el glosario.

1 Cómo se conectan HTML, CSS y JavaScript

HTML — Estructura (qué hay)

Define el **contenido y la estructura**: encabezados, párrafos, tablas, botones, inputs. Es texto plano con etiquetas. No decide cómo se ve.

```
<!-- productos.html -->
<div class="table-filter-bar">
  <button class="edit">Editar</button>
</div>
```

CSS — Presentación (cómo se ve)

Define **estilos**: colores, tamaños, espacios, alineación, animaciones. Selecciona elementos del HTML y aplica reglas (**propiedad: valor**).

```
.edit {
  background: linear-gradient(135deg, #facc15, #eab308);
  border: none;
  border-radius: 8px;
}
```

JavaScript — Comportamiento (qué pasa)

Lee y modifica el **DOM**: reacciona a clicks, trae datos, crea/borra elementos. Sin JS un botón no hace nada.

```
document.querySelector('.edit')
  .addEventListener('click', () => { ... });
```

Angular — El framework que coordina todo

Un componente = **plantilla HTML** + **estilos CSS** + **clase TypeScript**. La plantilla extiende HTML con directivas de Angular que manipulan el DOM por ti (de forma declarativa).

```
<button
  *ngIf="p.hasHistory" <!-- directiva -->
  (click)="deactivateProduct(p.id)">
  Dar de baja
</button>
```

El flujo en el navegador: ① Se descarga el HTML. ② Mientras se lee, se construye el **DOM** (árbol de elementos) y, con las hojas CSS, el **CSSOM**. ③ DOM + CSSOM se combinan en el **árbol de render**. ④ El motor calcula la geometría de cada caja (**layout**). ⑤ Pinta píxeles (**paint**). El resultado lo ves en pantalla. Angular (con **zone.js**) detecta cambios y actualiza el DOM automáticamente.

En la terminal de entrevista suele caer esta pregunta con ejemplos: «¿Cuál es la diferencia entre HTML, CSS y JS?» → Estructura / Presentación / Comportamiento. Sobre un típico: «¿El DOM es el HTML?» → No: el HTML es el origen, el DOM es la representación viva en memoria.

2 El DOM (Document Object Model)

Qué es

El **DOM** es la representación del documento HTML como un **árbol de nodos en memoria**, creada por el navegador al cargar la página. Es una **API programática**: a través de ella JavaScript (o Angular) puede leer y modificar elementos, estilos, atributos, contenido y eventos.

HTML ≠ DOM

Cuando el navegador acaba de leer **index.html**, construye el DOM **igualando** su contenido. Pero son cosas distintas: el **HTML es texto estático** (la fuente en el servidor); el **DOM es un objeto vivo**. Si JavaScript añade un **<div>**, el HTML en disco NO cambia: solo cambia el DOM de esa pestaña.

El árbol: nodos, padres e hijos

```
html
├── body
│   ├── app-root      <-- raíz de la app Angular
│   │   ├── .layout
│   │   │   ├── app-sidebar      hijo (1º)
│   │   │   ├── main.content     hermano del sidebar
│   │   │   └── .table-card      descendiente / hijo de main
│   └── ...
```

- **Nodo:** cualquier punto del árbol (elemento, texto, comentario, atributo).
- **Elemento:** un nodo etiquetado: `<div>`, `<table>`...
- Relaciones: `parentNode` (padre), `childNodes` (hijos), `nextSibling` (hermano), `firstChild`, `lastChild`.

API típica (JavaScript puro)

- `getElementById('x')` · `querySelector('.edit')`
- `createElement('div')` · `appendChild(node)`
- `removeChild(node)` · `innerHTML` · `textContent`
- `classList.add/remove/toggle` · `style.propiedad`
- `addEventListener('click', fn)`

En Angular se llama "templating"

- `(click)="..."` → registra un listener en el nodo.
- `*ngIf / *ngFor` → añade/quita/repite nodos.
- `[src]="..."` → asigna una propiedad del nodo.
- `[class.collapsed]` → toggla una clase CSS.
- `[(ngModel)]` → sincroniza valor del input ida y vuelta.

Render pipeline (cómo pasa el DOM a la pantalla)

1. **Parsing:** HTML → DOM · CSS → CSSOM.
2. **Render Tree:** se unen DOM + CSSOM, descartando lo invisible (`display:none`).
3. **Layout / Reflow:** se calculan tamaños y posiciones de cada caja.
4. **Paint:** se rellenan píxeles (colores, bordes, imágenes).
5. **Composite:** se unen las capas y se muestran.

Reflow vs repaint: cambiar `width` o `position` obliga a recalcular layout (más caro). Cambiar solo `color` o `background` solo repinta. Por eso animar `transform` / `opacity` (como el `fade-in-up` del proyecto) es más barato que animar `width` / `margin`.

Conceptos que suelen preguntar: **DOM vs Virtual DOM** (el DOM virtual es una copia en memoria que librerías como React usan para difuminar cambios y actualizar el DOM real en lote; Angular clásico usa change detection sobre el DOM real, con `zone.js`). **Shadow DOM** (encapsulación de estilos; Angular Material lo usa, y Angular emula encapsulación con atributos `_ngghost` / `_ngcontent`).

3 HTML: etiquetas y atributos usados en el proyecto

3.1 Estructura mínima de un documento

```
<!-- src/index.html del proyecto -->
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8"> <!-- codificación de caracteres -->
  <title>Ventas</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <app-root></app-root> <!-- aquí Angular monta toda la app -->
</body>
</html>
```

3.2 Etiquetas usadas en Ventas-Frontend (glosario)

Etiqueta	¿Qué es?	Uso real en el proyecto
<code><div></code>	Caja contenedora genérica (sin semántica), bloque por defecto. El elemento más usado.	Layouts: <code>.layout</code> , <code>.main-grid</code> , <code>.table-card</code> , <code>.table-actions</code>
<code></code>	Contenedor en línea (sin salto de línea). Ideal para badges y píldoras.	Badges de pago: <code></code>

Etiqueta	¿Qué es?	Uso real en el proyecto
<main>	Contenido principal único de la página (semántico, accesible).	<main class="content"> en todos los módulos
<nav>	Sección de navegación (semántico).	<nav class="nav"> dentro del sidebar
<aside>	Contenido secundario/complementario (semántico).	<aside> para el sidebar de la app
<h1>...<h4>	Encabezados jerárquicos (un único h1 por página).	<h1> en login, <h2> en títulos de módulo, <h3> en secciones de Reportes
<p>	Párrafo de texto.	Placeholders y textos de ayuda del POS
	Texto importante en negrita (semántico).	Totales y resúmenes: en el cobro y reportes
/	Lista no ordenada con sus ítems (bloque).	Menú del sidebar
	Imagen (etiqueta de reemplazo: apunta a un recurso, no lo contiene). Vacía: sin cierre.	 (imágenes de productos)
<a>	Enlace/hipervínculo. Puede llevar href o routerLink (Angular).	Links del menú y "olvidé mi contraseña"
<button>	Botón clickeable. Por defecto display:inline-block.	Acciones Editar/Eliminar/Dar de alta, guardar formularios
<input>	Campo de entrada (texto, número, fecha, archivo... según type), Vacía.	<input type="file">, <input type="date">, precios/stock
<label>	Etiqueta de un campo: al hacer clic enfoca el input asociado (for → id).	<label for="filterCategory"> sobre los filtros
<select> / <option>	Lista desplegable con sus opciones.	Filtros por categoría/rol/método de pago, roles al crear usuario
<table>	Tabla de datos.	Tablas CRUD (productos, usuarios, ventas, compras...)
<thead> / <tbody>	Agrupar filas de cabecera / cuerpo de una tabla.	<thead> con los títulos de columnas y <tbody> con los datos
<tr>	Fila de tabla.	Una fila por elemento en el *ngFor
<th>	Celda de cabecera (negrita por defecto, semántica).	<th>Nombre</th>, <th>Precio</th> ...
<td>	Celda de datos.	Cada valor del registro; con colspan="8" en la fila "sin datos"
<canvas>	Lienzo para dibujar gráficos (JS lo pinta).	Gráficas de Reportes con Chart.js

Etiquetas presentes en los templates de la app (conteo real): div(215) td(94) button(69) th(65) tr(49) span(49) input(40) label(35) strong(30) option(19) table(14) h2(13) h3(13) thead(13) tbody(13) img(12) p(10) main(10) select(9) li(5) canvas(4) a(4) h4(3) ul(1) aside(1) nav(1).

3.3 Atributos HTML usados (glosario)

Atributo	¿Qué hace?	Uso real
class	Asigna una o varias clases CSS para seleccionar/estilizar (el más usado: 357 veces).	class="table-filter-bar"
id	Identificador único de un elemento (para CSS #id y para label[for]).	id="filterCategory" enlazado al label
type	Define el comportamiento del input: text, number, password, date, file, button/submit ...	Precio: <input type="number">
name	Clave del campo (importante al enviar formularios).	Campos de login
value	Valor inicial de un input/option.	Valores por defecto en opciones
placeholder	Texto de ayuda que desaparece al escribir.	Buscar producto por nombre/barcode en el POS
min / max	Límites numéricos mínimos/máximos (HTML constraint para input numérico).	min="1" en precio, min="0" en stock
maxLength	Máximo de caracteres de un texto.	Límites de SKU/nombre en formularios
pattern	Expresión regular que valida el valor del campo (validación nativa).	Validaciones de contraseñas
accept	Filtra los tipos de archivo permitidos en un input type="file".	accept="image/*" al subir foto de producto
inputmode	Teclado que se muestra en móvil (numérico, decimal...).	Campos de montos
src	Ruta del recurso de una imagen/script (etiqueta de reemplazo).	[src]="imageUrl(p.img)"
alt	Texto alternativo de una imagen (accesibilidad y SEO; si falla la carga, se muestra).	alt="{{ p.name }}"
href	Destino de un enlace. Puede ser URL o fragmento.	
rel	Relación del enlace con la página (stylesheet, noopener ...).	rel="stylesheet"

Atributo	¿Qué hace?	Uso real
<code>for</code>	Asocia un label a un input por su id.	Labels de los filtros
<code>colspan</code> / <code>rowspan</code>	Expande una celda a varias columnas / filas.	<code>colspan="8"</code> en fila "No hay resultados"
<code>title</code>	Tooltip con información adicional.	Ayudas en botones del POS
<code>lang</code>	Idioma del documento (en <code><html></code>).	<code><html lang="es"></code>
<code>meta charset</code> / <code>content</code>	Metadatos del documento (codificación, viewport...).	<code>charset="UTF-8"</code>

3.4 Bonus: directivas y bindings de Angular en las plantillas

Las plantillas usan sintaxis de Angular sobre el mismo HTML. El navegador ve el resultado final (ya sin directivas).

Sintaxis	Qué hace	Equivalente a JS/API DOM
<code>*ngIf="cond"</code>	Añade/elimina el elemento y su subárbol según la condición.	crear/borrar nodos
<code>*ngFor="let p of lista"</code>	Repite el elemento por cada ítem de la lista (tablas).	bucle que crea nodos
<code>*hasPermission="'PERMISO'"</code>	Directiva propia del proyecto: muestra si el usuario tiene el permiso.	if + ocultar
<code>(click)="metodo(\$event)"</code>	Evento del DOM → ejecuta método del componente.	<code>addEventListener</code>
<code>[src]</code> / <code>[value]</code>	Binding de propiedad: valor del TS → nodo DOM.	<code>element.src = valor</code>
<code>[(ngModel)]="campo"</code>	Two-way: input → modelo y modelo → input.	<code>value</code> + 'input' event
<code>[disabled]</code>	Binding de la propiedad disabled del botón.	<code>el.disabled = bool</code>
<code>[ngValue]</code>	Valor real (número/objeto) de un option, no string.	opción seleccionada
<code>[class.collapsed]="cond"</code>	Toggles una clase CSS según condición.	<code>classList.toggle</code>
<code>(ngModelChange)</code>	Evento al cambiar el modelo (resetea la paginación).	event 'change'
<code>routerLink</code>	Navegación interna SPA sin recargar página.	<code>location.change</code>
<code> paginate: page : pageSize</code>	"Pipe" (transformación) que recorta la lista a una página.	<code>array.slice</code>

4 CSS: fundamentos

4.1 Sintaxis y cómo se aplica

```
selector {
  propiedad: valor;
  propiedad: valor;
}
```

- **Selector:** a qué elementos se aplica (clase, etiqueta, id, pseudo-clase...).
- **Declaración:** propiedad : valor ; (termina en punto y coma).
- **Regla:** el selector + sus declaraciones.

En Angular cada componente trae su propia hoja en `styleUrls` (EJS: `styleUrl: './productos.css'`) y hay estilos globales en `src/styles.css`. Angular **aisla** los estilos del componente (encapsulación) añadiendo atributos internos; por eso una clase `.edit` de productos no toca la de usuarios.

4.2 La cascada: origen, especificidad y orden

Si varias reglas aplican al mismo elemento, la ganadora se decide por (en orden):

1. **Importancia:** `!important` vence (usar con cautela).
2. **Especificidad** del selector: a mayor puntaje gana.
3. **Orden de aparición:** si empatan, gana la última en el archivo.

Calcular especificidad (sistema de tuplas)

Selector	Inline	#id	.clase/:pseudo	etiqueta::pseudo-el	Total
<code>.edit</code>	0	0	1	0	(0,0,1,0)
<code>.table-card th</code>	0	0	1	1	(0,0,1,1) → gana a .edit
<code>.table-filter-field select.filter-select</code>	0	0	2	1	(0,0,2,1)

Consejo de entrevista: **la especificidad se compara de izquierda a derecha**; en la práctica el proyecto casi solo usa clases, así que el "empate" se resuelve por orden y por no abusar de `!important`

4.3 Herencia

Unas propiedades **se heredan** de padre a hijo (tipografía, `color`, `text-align`) y otras **no** (caja: `margin`, `padding`, `border`, `background`, `width/height`, `display`, `position`). El valor especial `inherit` fuerza la herencia y `initial` vuelve al valor por defecto.

4.4 Selectores usados en el proyecto

Selector	Qué selecciona	Ejemplo real
<code>.clase</code>	Elementos con esa clase (el más usado).	<code>.sidebar</code> , <code>.table-card</code>
<code><etiqueta></code>	Elementos por etiqueta.	<code>td</code> , <code>th</code> , <code>input</code>
<code>.padre .hijo</code> (descendiente)	Un hijo dentro de .padre (cualquier profundidad).	<code>.table-card tbody tr</code>
<code>.a.b</code> (aditivo)	Elemento que tiene a la vez ambas clases.	<code>.main-grid > .table-container:only-child</code>
<code>elemento: hover</code> (pseudo-clase)	Estado "mouse encima".	<code>.edit: hover</code> , <code>.nav a: hover</code>
<code>: focus</code>	Elemento con foco de teclado (inputs).	<code>input: focus</code> con borde azul
<code>: nth-child(even)</code>	Elementos en posición par (cebra de tablas).	<code>tr: nth-child(even)</code> en filas alternas
<code>: last-child</code> / <code>: only-child</code>	Último / único hijo.	<code>th: last-child</code> , <code>.table-container: only-child</code>
<code>:: before</code> / <code>:: after</code>	Elementos generados por CSS (no existen en el DOM).	Barras de acento de las tarjetas de Reportes
<code>:: -webkit-scrollbar...</code>	Personaliza el scrollbar en Chrome/Edge.	Scrollbar oscuro de <code>.table-scroll</code>
<code>:: -webkit-calendar-picker-indicator</code>	Icono del date picker nativo.	<code>filter: invert(0.8)</code> para el tema oscuro
<code>[attr=...]</code> (atributo)	Elementos con un atributo/valor concreto.	No usado hoy

4.5 El modelo de caja (Box Model) — pregunta obligada



content-box (por defecto)

`width` = solo el contenido. Padding y border **se suman** → el elemento ocupa más de lo que escribiste (fácil de romper layouts).

border-box (lo que usa la app)

`width` = contenido + padding + border. **El tamaño declarado es el total**. Por eso el proyecto aplica `box-sizing: border-box` a sus inputs.

```
.form-card input { box-sizing: border-box; height: 38px; }
```

Esquema para responder en entrevista: "Hay 4 capas de dentro hacia fuera: content → padding (espacio interno) → border (borde) → margin (separación exterior). Con `box-sizing: border-box` el ancho/alto declarado incluye padding y border, lo que hace los layouts predecibles."

4.6 Unidades de medida usadas

Unidad	Tipo	Qué mide	Uso en el proyecto
px	Absoluta	Píxel físico abstracto (no escala con el texto del usuario).	La más usada (625 ocurrencias): paddings, tamaños, bordes
%	Relativa	Porcentaje del bloque contenedor (padre) según el eje.	<code>width:100%</code> , <code>min-width:0</code>
vh / vw	Relativa al viewport	1vh = 1% del alto de la ventana; 1vw = 1% del ancho.	<code>height:100vh</code> (layout de pantalla completa)
rem	Relativa al <code>font-size</code> de <code><html></code>	Escala la tipografía global; accesible.	Textos base de tablas y tarjetas
em	Relativa al <code>font-size</code> del elemento actual	Se propaga con la herencia/cadena.	<code>letter-spacing</code> y márgenes pequeños
fr	Grid	Fracción del espacio libre en CSS Grid.	<code>grid-template-columns: 350px 1fr</code> (columna flexible)
s / ms	Tiempo	Segundos / milisegundos (transiciones, animaciones).	<code>transition: 0.3s</code> , <code>250ms</code>
deg	Ángulo	Grados (rotación y gradientes).	<code>linear-gradient(135deg, ...)</code>

4.7 Colores

- **Hex** `#RRGGBB`: el formato dominante en la app (`#1e293b` gris pizarra de tarjetas, `#0f172a` fondo, `#3b82f6` azul primario, `#facc15` amarillo de "Editar").
- **rgba(r, g, b, alfa)**: color con transparencia (`box-shadow: 0 4px 20px rgba(0,0,0,0.25)`).
- **Gradientes**: `background: linear-gradient(135deg, #facc15, #eab308)` → de un color a otro en diagonal. Los usan los botones de acción (Editar amarillo, Eliminar rojo, Dar de baja naranja).
- También hay `color-scheme`, `hsla` en Material y variables Material SCSS.

4.8 display: los valores clave (memorizar)

Valor	Qué hace	Ejemplo
<code>block</code>	Ocupa todo el ancho, salto de línea antes/después.	<code>div</code> , <code>p</code> , <code>h2</code> (por defecto)
<code>inline</code>	Fluye en la misma línea; ignora width/height y márgenes verticales.	<code>span</code> , <code>a</code> , <code>strong</code>
<code>inline-block</code>	En línea pero acepta width/height/padding.	botones, badges
<code>flex</code>	Contenedor flexbox: alinea hijos en fila/columna.	<code>.layout</code> , <code>.table-actions</code>
<code>grid</code>	Contenedor grid de 2 ejes con columnas/filas explícitas.	<code>.main-grid</code>
<code>none</code>	El elemento NO se renderiza: no ocupa espacio y no es accesible.	Sidebar colapsado oculta textos

Las tres formas de "ocultar" algo se caen en entrevistas: `display:none` (quita del flujo y no se pinta), `visibility:hidden` (ocupa espacio pero invisible) y `opacity:0` (invisible pero sigue interactuable y ocupa espacio).

5 Glosario de propiedades CSS usadas en el proyecto

Cada tabla agrupa una categoría. «En el proyecto» muestra un ejemplo real de `src/styles.css` o de los CSS de componentes.

5.1 Layout: Flexbox, Grid y posicionamiento

Propiedad	Qué hace	En el proyecto
<code>display: flex</code>	Activa Flexbox: los hijos directos se colocan en fila (eje principal) y se puede alinear/distribuir.	<code>.layout</code> (caja a pantalla completa), <code>.table-actions</code> (botones por fila)
<code>display: grid</code>	Activa CSS Grid: filas y columnas explícitas con tracks.	<code>.main-grid</code> : <code>grid-template-columns: 350px 1fr</code>
<code>flex-direction</code>	Eje principal de un contenedor flex: <code>row</code> (por defecto) o <code>column</code> .	<code>.sidebar</code> y <code>.content</code> apilan en columna
<code>justify-content</code>	Alinea los ítems a lo largo del EJE PRINCIPAL (horizontal en row, vertical en column).	<code>.table-actions { justify-content: center }</code> , <code>.menu-title { justify-content: center }</code> . Valores: <code>flex-start/center/flex-end/space-between/space-around</code> .

Propiedad	Qué hace	En el proyecto
<code>align-items</code>	Alinea los ítems en el EJE CRUZADO (vertical en row).	<code>.user-box { align-items: center }</code> , <code>.table-filter-bar { align-items: flex-end }</code> (filtros pegados abajo)
<code>align-self</code>	Anula <code>align-items</code> SOLO para un ítem.	<code>.deactivated-link { margin-left: auto; align-self: flex-end }</code>
<code>flex-wrap</code>	Permite que los ítems bajen de línea (<code>wrap</code>) o se compriman (<code>nowrap</code>).	<code>.table-filter-bar { flex-wrap: wrap }</code> , <code>.table-actions { flex-wrap: wrap }</code>
<code>flex: 1</code>	Shorthand: <code>flex-grow flex-shrink flex-basis</code> . <code>flex:1</code> = crece para ocupar el hueco restante.	<code>.content { flex:1 }</code> , <code>.nav { flex:1 }</code> , <code>.table-scroll { flex:1 }</code>
<code>flex-shrink</code>	Capacidad de encogerse cuando falta espacio (<code>0</code> = no encogerse).	<code>.user-box, .menu-title { flex-shrink: 0 }</code>
<code>gap</code>	Separación entre ítems flex/grid (sin márgenes por elemento).	<code>gap: 8px</code> en <code>.table-actions</code> , <code>gap: 30px</code> en el grid, <code>gap: 12px</code> en filtros
<code>grid-template-columns</code>	Define las columnas del grid (anchuras).	<code>grid-template-columns: 350px 1fr</code> → columna fija + columna flexible
<code>grid-template-rows</code>	Define las filas del grid.	Usado 1 vez en el layout de Reportes
<code>grid-column</code>	Ubica un ítem en columnas concretas; <code>1 / -1</code> = ocupar todas.	<code>.main-grid > .table-container:only-child { grid-column: 1 / -1 }</code>
<code>position: sticky</code>	Se comporta relativo hasta que al scrollear llega al <code>top</code> , y se "pega".	<code>.table-card th { position: sticky; top: 0 }</code> → cabecera fija
<code>position: relative / absolute / fixed</code>	Posicionamiento: relativo a sí mismo, al ancestro posicionado, o al viewport.	<code>position: relative</code> para anclar badges; <code>absolute</code> / <code>fixed</code> en modales
<code>top / right / bottom / left</code>	Desplazamiento desde los bordes (con position).	<code>th { top: 0 }</code>
<code>inset</code>	Shorthand de top/right/bottom/left.	Usado en un par de reglas de Márgenes
<code>z-index</code>	Orden de apilamiento en el eje Z (los mayores arriba).	<code>th { z-index: 2 }</code> para la cabecera sobre filas
<code>overflow / overflow-x / overflow-y</code>	Controla qué pasa si el contenido desborda: <code>hidden</code> (recorta), <code>auto</code> (scroll).	<code>.content { overflow: hidden }</code> , <code>.table-scroll { overflow: auto }</code>
<code>-webkit-overflow-scrolling: touch</code>	Scroll con inercia nativa en iOS.	Aplicado en <code>.table-scroll</code>

5.2 Modelo de caja (caja = box)

Propiedad	Qué hace	En el proyecto
<code>width / height</code>	Ancho / alto de la caja (según box-sizing).	<code>.sidebar { height: 100vh }</code> , <code>width: 40px</code> en avatar
<code>min-width / min-height</code>	Tamaño mínimo: no baja de ahí.	<code>.table-scroll { min-height: 0 }</code> (clave para scroll interno), <code>min-width: 480px</code> en tablas
<code>max-width / max-height</code>	Tamaño máximo.	<code>max-width: 340px</code> en tarjetas de formulario, <code>max-height: 100vh</code> en sidebar
<code>padding</code>	Espacio INTERNO entre contenido y borde (en sentido horario: top right bottom left).	<code>.table-card th { padding: 13px 14px }</code> (13 vertical, 14 horizontal)
<code>padding-top/right/bottom/left</code>	Padding por lado (unilateral).	<code>.table-scroll { padding-top: 20px }</code>
<code>margin</code>	Espacio EXTERNO alrededor de la caja (separa de hermanos).	<code>margin: 10px 0</code> en links del sidebar, <code>margin: 0 auto</code> para centrar
<code>margin-left: auto</code>	Colapsa el margen izquierdo: empuja el elemento a la derecha (truco clave).	<code>.deactivated-link { margin-left: auto }</code> → botón "Ver desactivados" a la derecha
<code>margin-top / left / right / bottom</code>	Margen unilateral.	<code>margin-bottom: 15px</code> , <code>margin-top: 10px</code>
<code>border</code>	Borde: <code>border: ancho estilo color</code> .	<code>border: 1px solid #263449</code> (borde de <code>.table-card</code>)
<code>border-radius</code>	Esquinas redondeadas (píxeles, % o 50% para círculos).	<code>border-radius: 12px</code> en tarjetas, <code>50%</code> en avatares
<code>border-bottom / border-top</code>	Borde solo de un lado (separadores).	<code>border-bottom: 1px solid #263449</code> bajo la barra de filtros
<code>border-color</code>	Color del borde por separado.	Focus de inputs: <code>border-color: #3b82f6</code>
<code>border-collapse: collapse</code>	Funde los bordes de celdas y filas adyacentes (tablas).	Aplicado a todas las tablas de <code>.table-card</code>

Propiedad	Qué hace	En el proyecto
<code>box-sizing: border-box</code>	<code>width/height</code> incluyen padding y border.	Inputs de formularios (evita desbordarse)

5.3 Tipografía y texto

Propiedad	Qué hace	En el proyecto
<code>font-family</code>	Familia tipográfica (lista de fuentes de fallback).	App usa Segoe UI/Arial; Material su fuente propia
<code>font-size</code>	Tamaño de la letra (px, rem, em...).	<code>font-size: 11px</code> en etiquetas de filtro, <code>13px</code> en botones, <code>14px</code> en tablas
<code>font-weight</code>	Grosor: <code>400</code> normal, <code>600/700</code> bold.	<code>font-weight: 700</code> en cabeceras (<code>th</code>) y sumarios
<code>font-variant-numeric: tabular-nums</code>	Dígitos de ancho fijo: los números no "tiemblan" al cambiar.	Columna de montos <code>.money</code> (ventas, etc.)
<code>line-height</code>	Altura entre líneas de texto.	Textos de párrafos de pantallas de login
<code>letter-spacing</code>	Espacio entre letras (tracking).	Etiquetas en mayúsculas: <code>letter-spacing: 0.6px</code>
<code>text-align</code>	Alineación horizontal del texto: left/center/right/justify.	<code>text-align: center</code> en cabeceras y celdas, <code>.money { text-align: right }</code>
<code>text-decoration</code>	Subrayado/tachado (none quita el subrayado de los enlaces).	<code>.sidebar a { text-decoration: none }</code>
<code>text-transform</code>	Transforma mayúsculas/minúsculas (uppercase, capitalize...).	<code>text-transform: uppercase</code> en encabezados de tabla
<code>white-space</code>	Controla saltos de línea y espacios: <code>nowrap</code> = una sola línea.	<code>white-space: nowrap</code> en celdas/botones (no romper)
<code>color</code>	Color del texto.	Texto <code>#94a3b8</code> (gris) sobre <code>#e2e8f0</code> (claro) en tablas

5.4 Fondo, bordes decorativos y apariencia

Propiedad	Qué hace	En el proyecto
<code>background</code>	Shorthand de color/imagen/posición/repetición. Acepta colores y gradientes.	<code>background: #0f172a</code> (fondo), <code>linear-gradient(135deg, #facc15, #eab308)</code> (botón Editar)
<code>background-size</code>	Tamaño del fondo si es imagen (cover, %, 200%...).	<code>background-size: 200% 100%</code> en el skeleton (efecto shimmer)
<code>background-position</code>	Posición de la imagen de fondo.	Animación del shimmer lo desplaza
<code>opacity</code>	Transparencia de TODA la caja (0–1). Si es 0, no se ve pero existe.	Filas placeholder: <code>opacity: 0.7</code> ; animación fade-in
<code>box-shadow</code>	Sombra exterior: <code>x y blur spread color</code> (varias sombras separadas por coma).	<code>box-shadow: 0 4px 20px rgba(0,0,0,0.25)</code> en tarjetas
<code>outline</code>	Anillo exterior (no afecta el tamaño; suele quitarse en focus custom).	<code>input:focus { outline: none; border-color: #3b82f6 }</code>
<code>appearance</code>	Quita el estilo nativo de los controles (selects, inputs).	<code>appearance: none</code> en los <code><select></code> del proyecto
<code>filter</code>	Filtros visuales: <code>invert, blur, grayscale, brightness...</code>	<code>filter: invert(0.8)</code> al icono del date picker
<code>cursor</code>	Forma del cursor (pointer sobre elementos clickeables).	<code>cursor: pointer</code> en botones, selects y filas
<code>pointer-events</code>	Si el elemento recibe eventos del mouse (none = transparente al clic).	Usado en overlays de modales
<code>visibility</code>	visible/hidden: oculta pero conserva el espacio.	Acompaña algunas transiciones del submenú
<code>color-scheme</code>	Indica al navegador la paleta (claro/oscuro) para controles nativos.	Definido en el tema oscuro de la plataforma
<code>content</code>	Pseudo-elementos <code>::before/::after</code> : inserta texto/forma generado por CSS.	Barras de acento de las tarjetas resumen de Reportes

5.5 Transformación, transición y animación

Propiedad	Qué hace	En el proyecto
<code>transform</code>	Transforma la caja en el plano (2D): <code>translateY()</code> , <code>scale()</code> , <code>rotate()</code> ...	<code>translateY(8px)</code> (animación de entrada), <code>scale(1.05)</code> al pasar el mouse por botones

Propiedad	Qué hace	En el proyecto
<code>transition</code>	Clave: <code>propiedad duración curva [delay]</code> . Anima los cambios de esa propiedad.	<code>transition: width 800ms ease-in-out, padding 800ms ease-in-out</code> en <code>.sidebar</code> ; <code>transition: all 0.25s ease</code> en botones
<code>animation</code>	Ejecuta keyframes: <code>nombre duración curva [iteraciones] [fill-mode]</code> .	<code>animation: fade-in-up 250ms ease-out forwards</code> en tablas; <code>animation: shimmer 1.2s infinite</code>
<code>@keyframes</code>	Define fotogramas con <code>from {...} to {...}</code> o <code>%</code> .	<code>fade-in-up</code> (opacity 0→1 y translateY 8px→0) y <code>shimmer</code>

5.6 Contenido de medios e imágenes

Propiedad	Qué hace	En el proyecto
<code>object-fit: cover</code>	La imagen rellena la caja recortando (sin deformar) como <code>background-cover</code> .	<code>.img { height: 40px; width: 40px; object-fit: cover; border-radius: 6px }</code> (fotos de productos)

6 Ejemplos reales de la app explicados paso a paso

Ejemplo A — El layout completo: Flexbox + viewport + colapso del menú

```
.layout /* contenedor flex a pantalla completa */ {
  display: flex;
  height: 100vh;
  background: #0f172a;
}
.sidebar /* hijo 1: fijo */ {
  width: 220px;
  transition: width 800ms ease-in-out; ← suave
}
.content /* hijo 2: se estira */ { flex: 1; }
```

- flex:** el sidebar y el contenido van lado a lado; `.content` con `flex:1` ocupa todo el hueco restante.
- 100vh:** el layout mide exactamente la altura de la ventana (no desborda la pantalla).
- Al colapsar el menú, Angular añade la clase `.collapsed` al contenedor (`[class.collapsed]='!sidebar.menuOpen'`) y la regla `.layout.collapsed .sidebar { width: 100px }` reduce el ancho; la **transition** anima ese cambio durante 800ms con curva `ease-in-out` (suave al inicio y al final).

Ejemplo B — CSS Grid: formulario fijo + tabla flexible

```
.main-grid {
  display: grid;
  grid-template-columns: 350px 1fr; /* 350px + resto */
  gap: 30px;
}
.main-grid > .table-container:only-child {
  grid-column: 1 / -1; /* ocupa las 2 columnas */
}
```

- Dos columnas: la tarjeta de formulario mide 350px; la tabla se lleva el **1fr** (fracción del espacio libre).
- Si el usuario **solo ve** (sin permisos de crear/editar), la tabla es el único hijo del grid y `:only-child` la expande con `grid-column: 1 / -1` — un truco con pseudo-clase que resuelve un bug de ancho.
- `min-width: 0` en la tabla permite que el grid la encoja y active el scroll horizontal en vez de desbordarse.

Ejemplo C — Barra de filtros: flex + margin-left:auto

```
.table-filter-bar {
  display: flex;
  align-items: flex-end; /* etiquetas encima de cada select */
  flex-wrap: wrap;
  gap: 12px;
}
.deactivated-link { margin-left: auto; } ← empuja el botón a la derecha
```

- `margin-left: auto` absorbe todo el espacio disponible a la izquierda del botón → el botón queda **pegado al borde derecho**. Truco omnipresente en los CRUD para el botón "Ver desactivados".
- `align-items: flex-end` mantiene bottom alineado con los selects; `flex-wrap` saca los controles a otra línea si no caben.

Ejemplo D — Cabecera de tabla pegajosa (sticky) y cebra

```
.table-card th {
  position: sticky; top: 0; z-index: 2;
}
.table-card tbody tr:nth-child(even) { background: #121f36; }
```

- `sticky + top:0`: la fila de títulos permanece visible al hacer scroll dentro de `.table-scroll`.
- `:nth-child(even)` pinta las filas pares (efecto cebra) para que las tablas sean legibles.

Ejemplo E — Animación de entrada con keyframes

```
@keyframes fade-in-up {
  from { opacity: 0; transform: translateY(8px); }
  to   { opacity: 1; transform: translateY(0); }
}
.fade-in { animation: fade-in-up 250ms ease-out forwards; }
```

- Al terminar la carga HTTP de una tabla, Angular añade la clase `fade-in` y el bloque "aparece" subiendo 8px y aclarándose en 250ms.
- `forwards` conserva el fotograma final (no rebota). Animar `opacity` + `transform` es barato para el navegador (solo paint/composite, sin reflow).

7 Preguntas típicas de entrevista (con respuestas cortas)

P: ¿Cuál es la diferencia entre `display:none`, `visibility:hidden` y `opacity:0`?

R: `display:none` quita el elemento del flujo y del render (no ocupa espacio). `visibility:hidden` lo oculta pero conserva su espacio (invisible e inerte). `opacity:0` lo hace transparente pero sigue presente, ocupando espacio y recibiendo eventos.

P: ¿Qué significa `box-sizing: border-box`?

R: Que en `width` / `height` ya van incluidos padding y border; el tamaño declarado es el total. Con `content-box` (default) se suman y el elemento puede desbordar sus contenedores.

P: ¿Cómo centras un div? (tres formas)

R: ① Flexbox: `display:flex; align-items:center; justify-content:center;` ② Grid: `display:grid; place-items:center;` ③ Automargin (solo horizontal con ancho fijo): `margin: 0 auto; width: 300px;`. Para centrar verticalmente con la vieja técnica: `position:absolute + transform`.

P: ¿Cuál es la diferencia entre posición `relative`, `absolute`, `fixed` y `sticky`?

R: `relative`: se desplaza respecto a su posición normal (sin sacarlo del flujo). `absolute`: sale del flujo y se ubica respecto al ancestro posicionado más cercano (o el body). `fixed`: se ubica respecto a la ventana (no se mueve al scrollear). `sticky`: relativo hasta que el scroll lo hace "pegarse" en un umbral (`top`). El proyecto usa sticky para las cabeceras de tabla.

P: ¿Cómo funciona la especificidad y qué gana si hay empate?

R: Se puntúa: `inline` > `#id` > `.clase/:pseudo` > etiqueta::`pseudo-elemento`. A igual puntaje gana la última regla en el CSS; `!important` rompe la especificidad (evitar).

P: ¿Qué es el DOM y qué no es?

R: Es el árbol de nodos en memoria que el navegador construye a partir del HTML y que JS/Angular manipulan. No es lo mismo que el HTML original: el HTML es la fuente; el DOM es el objeto vivo que puede cambiar en ejecución.

P: ¿Diferencia entre `textContent`, `innerHTML` y valor del `h1`?

R: `textContent` lee/escribe solo texto sin interpretar HTML (seguro). `innerHTML` parsea y renderiza HTML (peligroso con input de usuario → XSS). `nodeValue` lo usa en nodos de texto.

P: ¿Cuándo usarías `px`, `em`, `rem` o `%`?

R: `px` para tamaños fijos precisos; `rem` para tipografía accesible (escala el font-size raíz); `%` para anchos relativos al padre; `vh/vw` para layout de pantalla completa; unidades `fr` para grid. En esta app domina `px` + `%/vh` para el layout.

P: ¿Qué es el "Render pipeline"?

R: HTML → DOM, CSS → CSSOM, DOM+CSSOM → Render Tree, Layout (cajas), Paint (píxeles), Composite (capas). Cambios de geometría re-disponen layout (reflow); cambios de color solo repintan.

P: ¿HTML5: qué etiquetas semánticas conoces?

R: `header`, `nav`, `main`, `section`, `article`, `aside`, `footer`, `figure`, `mark`, `time` ... Mejoran accesibilidad y SEO. En el proyecto se usan `main`, `nav` y `aside`.

P: ¿Cómo se llama al cambio de estilo de un elemento al pasar el mouse, y se puede animar?

R: Pseudo-clase `:hover`. La transición se suaviza con `transition` (ej. `.edit:hover { transform: scale(1.05) }`).

P: ¿Qué es el margin collapsing?

R: Los márgenes verticales de elementos adyacentes (o padre/hijo sin borde ni padding) se "colapsan" y se usa el mayor en vez de sumarse. Uno de los bugs clásicos. (El proyecto evitó márgenes verticales en botones usando `gap` de flex.)

P: ¿Cómo combinas Angular y el DOM?

R: Angular genera/manipula el DOM mediante plantillas declarativas: `*ngIf` crea/borra nodos, `(click)` registra listeners, `[property]` sincroniza valores, y con `zone.js` detecta cambios y vuelve a renderizar lo necesario.

8 Cheat-sheet de un vistazo

display

`block` `inline` `inline-block` `flex` `grid` `none`

position

`static` `relative` `absolute` `fixed` `sticky`

Ocultar

`display:none` `visibility:hidden` `opacity:0`

Spec (de mayor a menor)

`inline > #id > .clase/:pseudo > etiqueta/::pseudo-el > heredado`.
Empate → orden en el archivo.

Centrar

`flex: align-items + justify-content` · `grid: place-items` ·
`margin: 0 auto`

Flexbox mínimo

`flex` (padre) + `justify-content` (eje principal) + `align-items` (cruzado) + `gap` (separación) + `flex:1` (crecer).

Caja (de dentro a fuera)

`content` → `padding` → `border` → `margin`. Con `box-sizing:border-box`, width incluye padding y border.

Unidades rápidas

`px` fijo `%` del padre `vh/vw` viewport `rem` raíz `em` propio `fr` grid

Truco del proyecto

`margin-left:auto` empuja un elemento al extremo derecho de un contenedor flex.

Paleta real de la app (memorízala para hablar concreto)

`fondo #0f172a` `tarjeta #1e293b` `primario #3b82f6` `amarillo #facc15 (editar)` `rojo #ef4444 (borrar)` `verde #22c55e (guardar/activo)`

Para repasar la noche antes: 1) memoriza el modelo de caja y `box-sizing`; 2) recuerda `display: block/inline/flex/grid` y cuándo usar cada uno; 3) practica centrar con flex y grid; 4) di con tus propias palabras qué es el DOM y el render pipeline; 5) ten 2 ejemplos reales de este proyecto (el layout flex con sidebar y la barra de filtros) para explicar conceptos "en vivo".